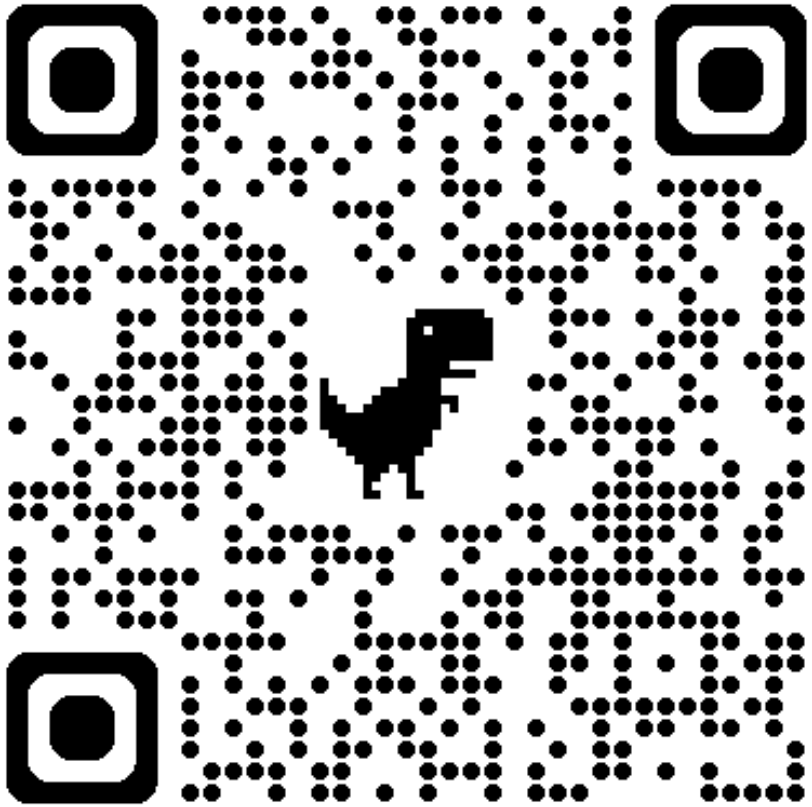


Introduction to GPU Acceleration

View the Slides



https://github.com/ResearchComputing/Intro_GPU_Acceleration

Meet the User Support Team



Layla Freeborn



Brandon Reyes



Andy Monaghan



Michael Schneider



John Reiland

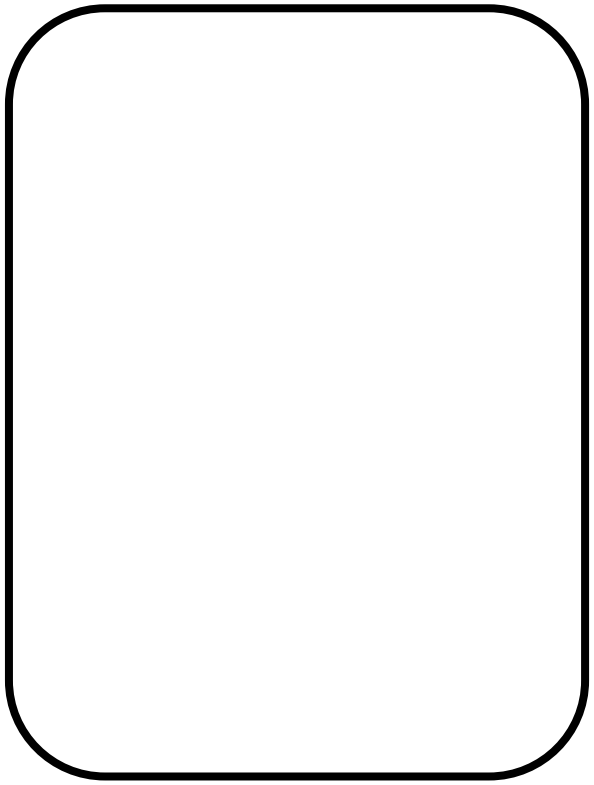


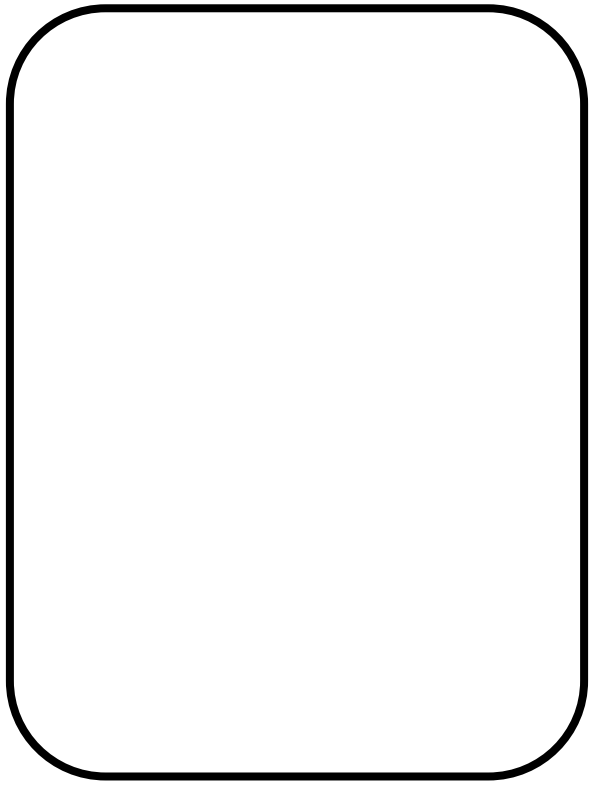
Dylan Gottlieb

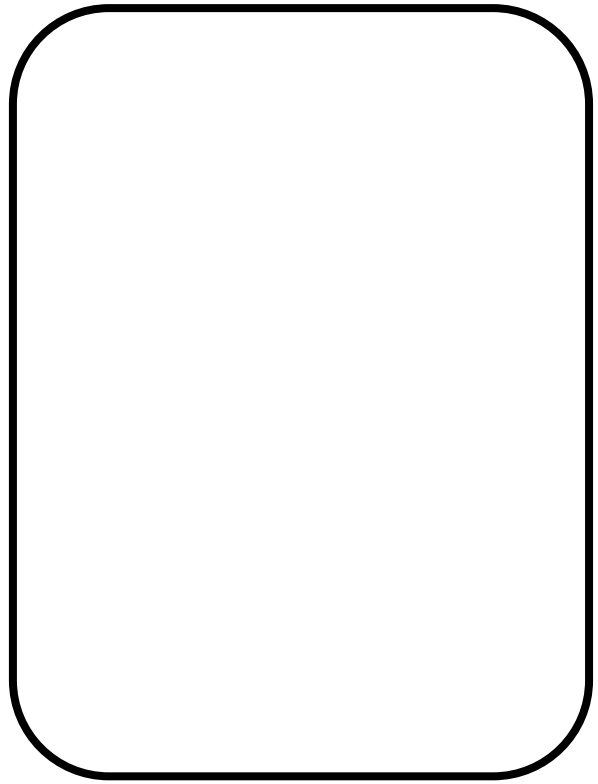


Mohal Khandelwal

Session Overview

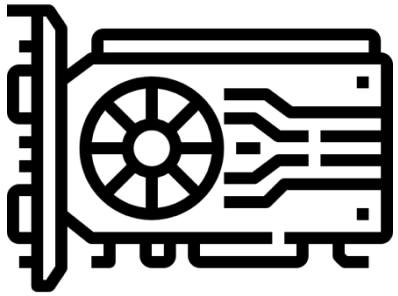






GPU Icon

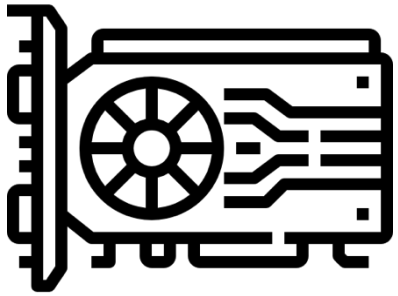
Session Overview



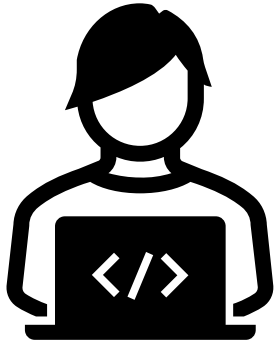
Basics Of GPUs

GPU Icon

Session Overview



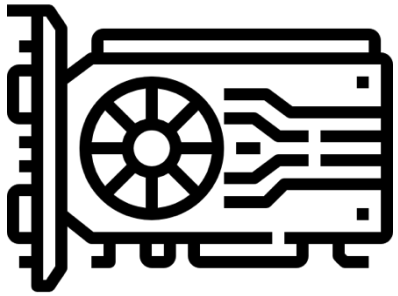
Basics Of GPUs



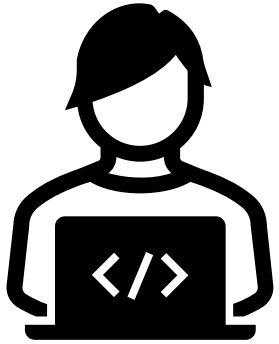
Code Optimization

GPU Icon

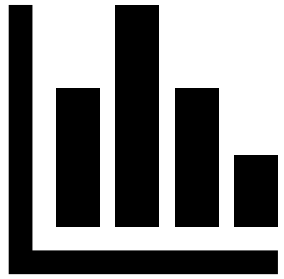
Session Overview



Basics Of GPUs



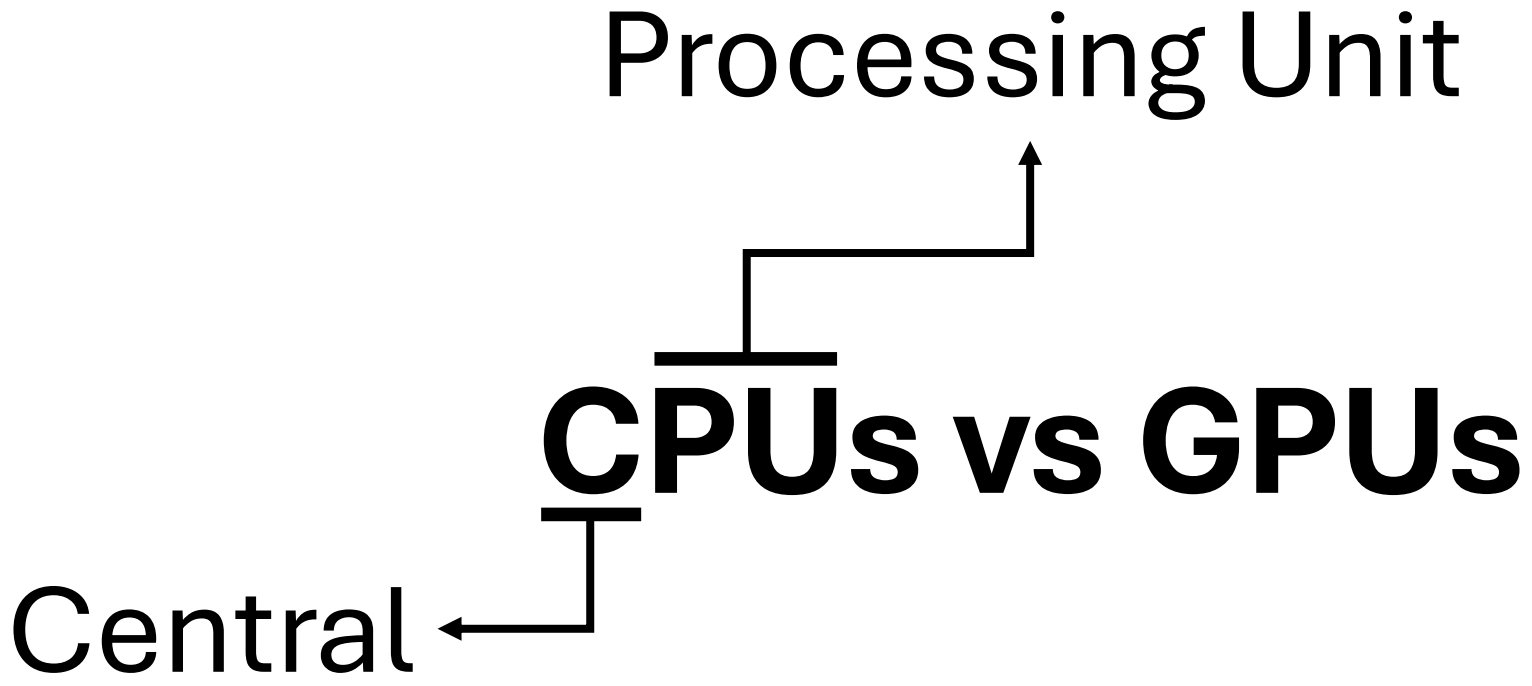
Code Optimization

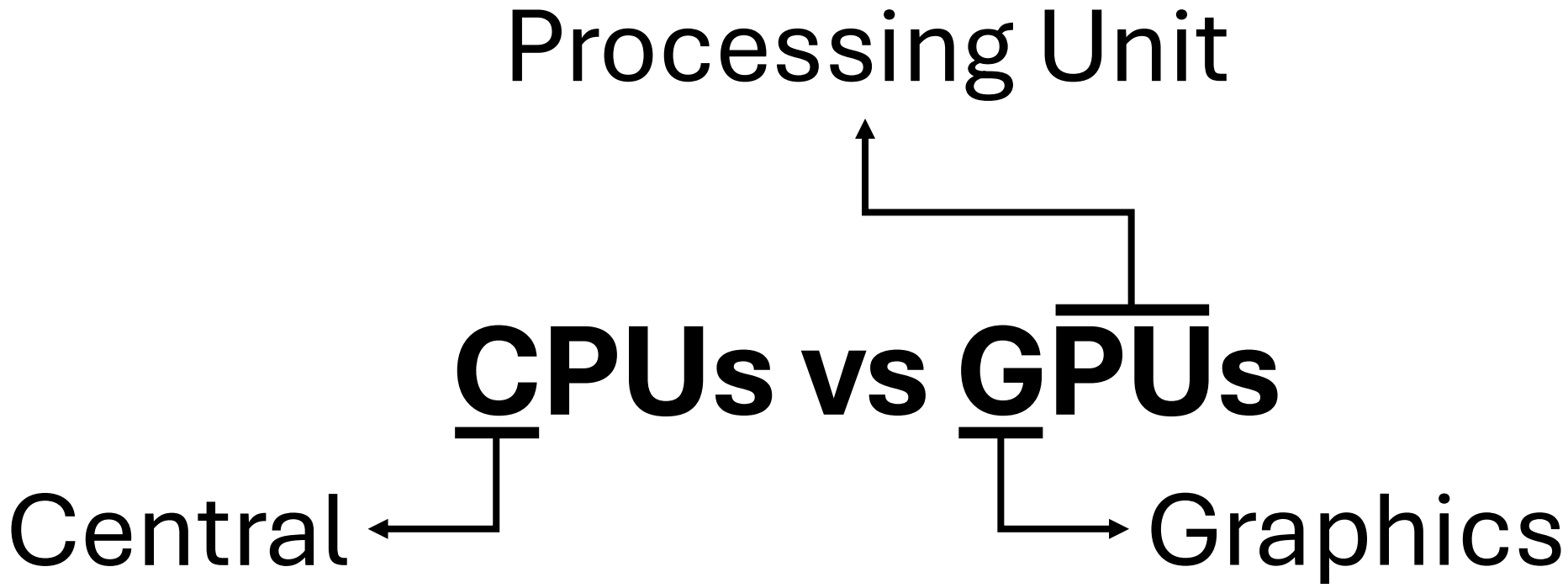


Monitoring GPU Usage

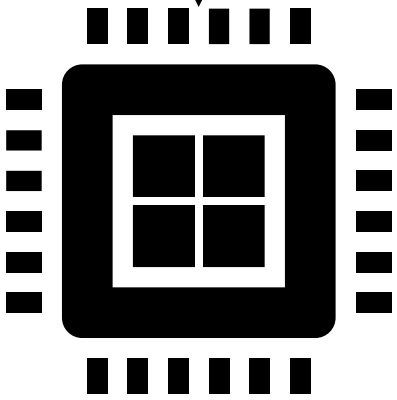
GPU Icon

CPUs vs GPUs

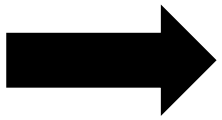

Processing Unit
CPUs vs GPUs
Central


Processing Unit
CPUs vs GPUs
Central
Graphics

CPUs vs GPUs

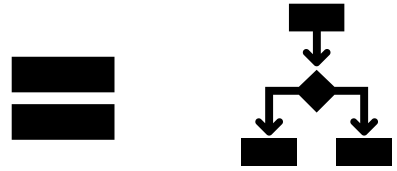


Dozens
Of Cores

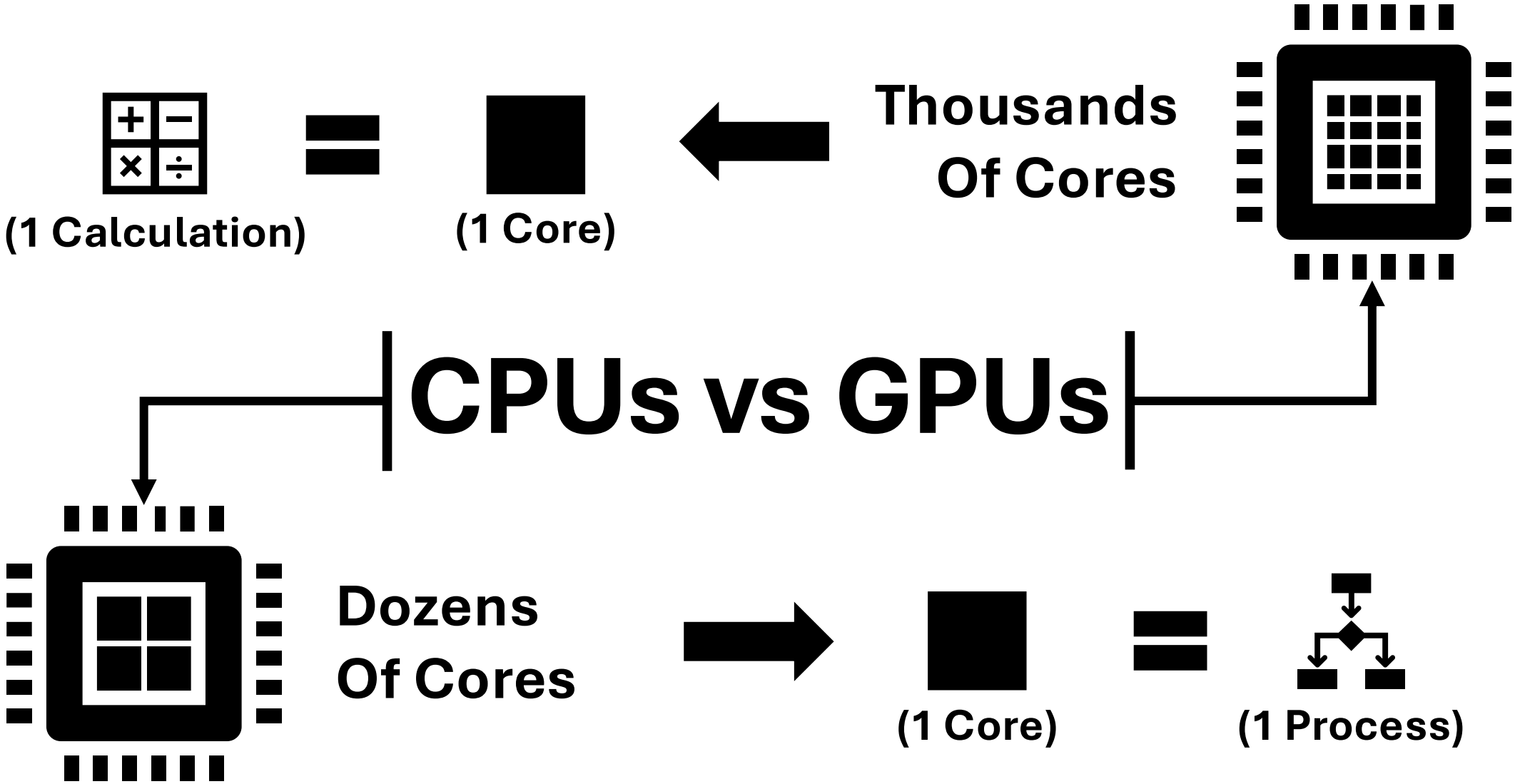




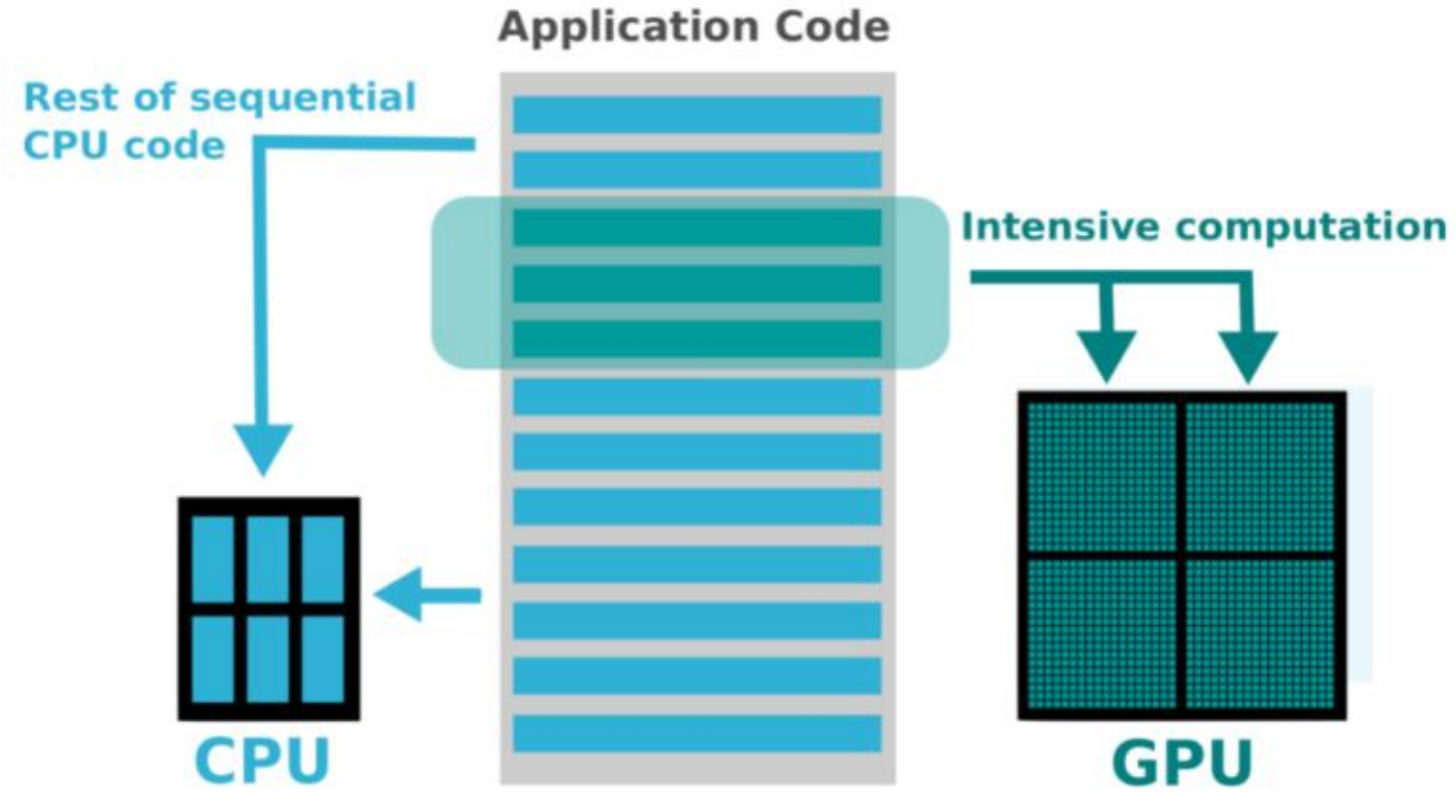
(1 Core)



(1 Process)


(1 Calculation)
(1 Core)
Thousands
Of Cores
CPUs vs GPUs
Dozens
Of Cores
(1 Core)
(1 Process)

Computational Offloading

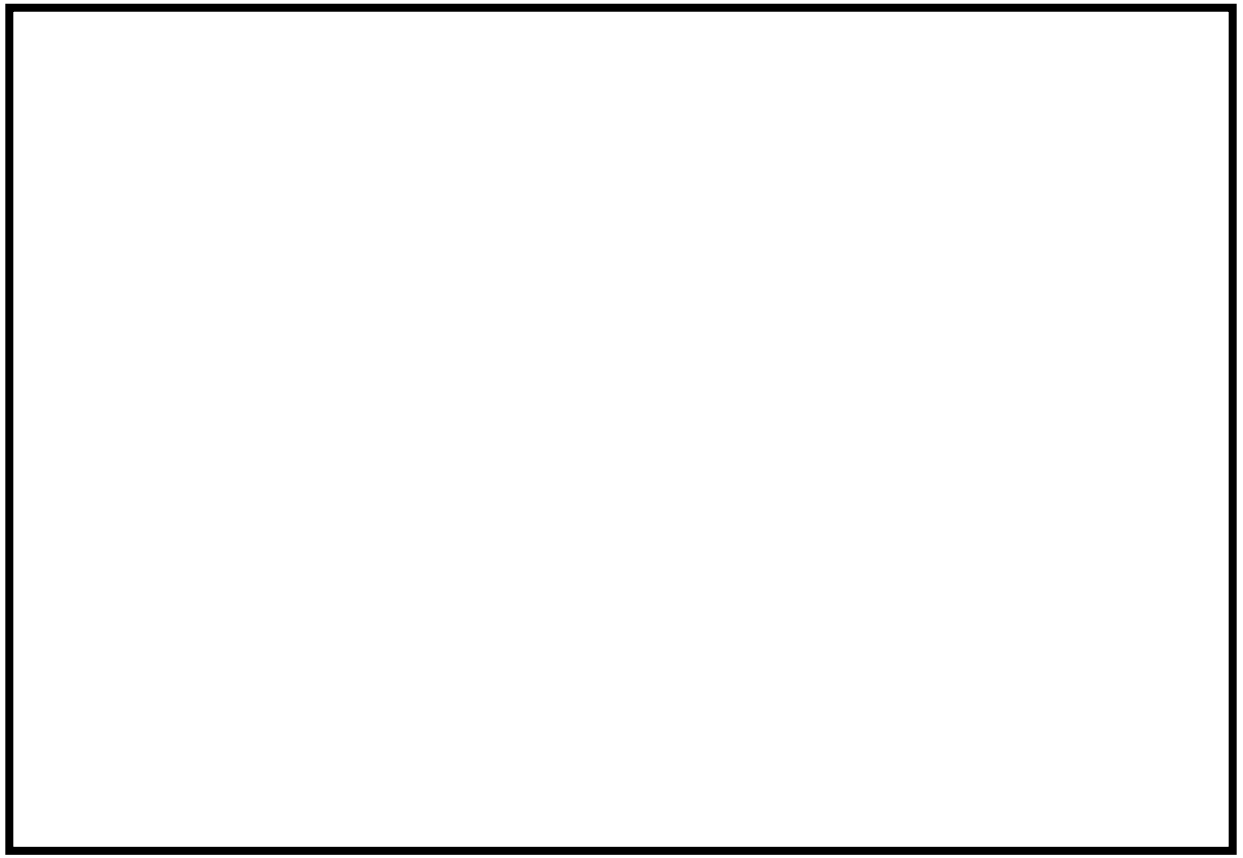


Graphic Source

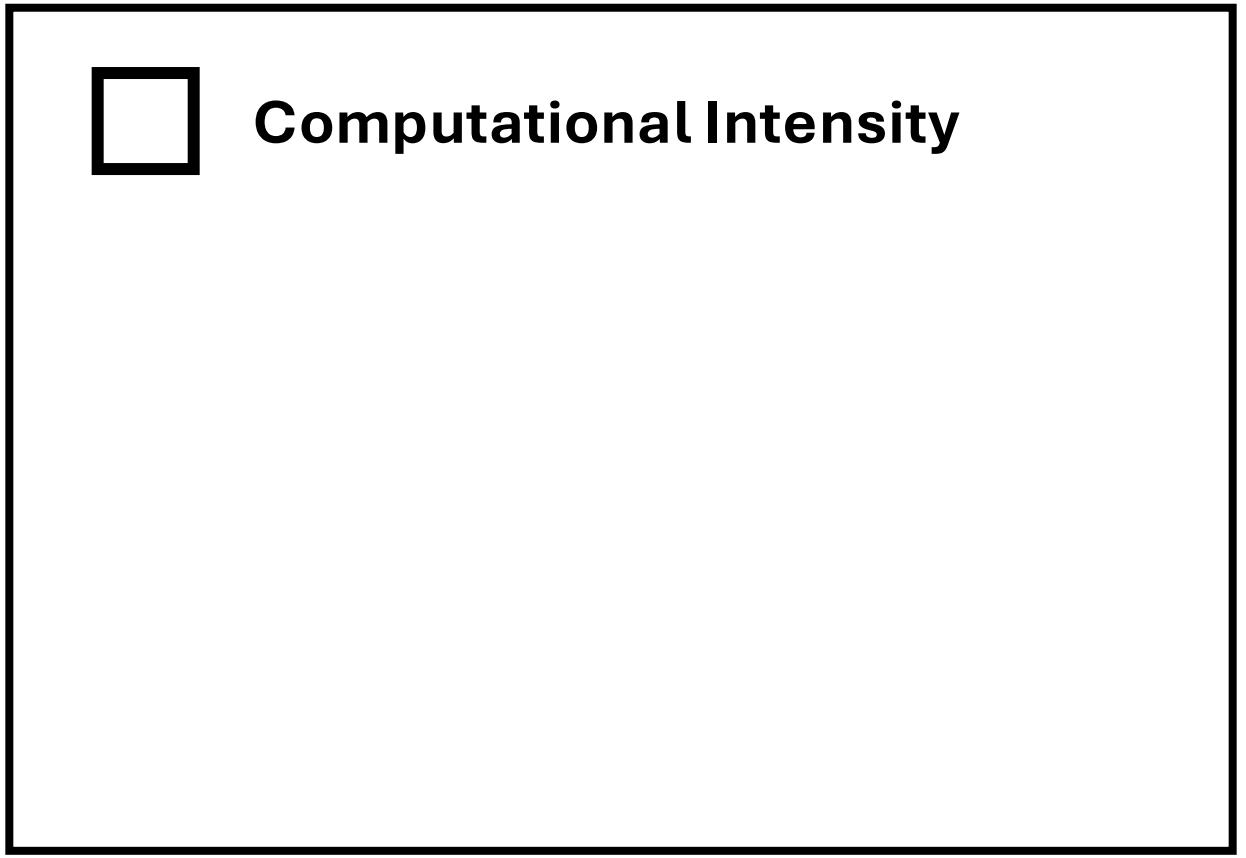
Criteria for GPU Acceleration

1. The time spent on computationally intensive parts of the workflow exceeds the time spent transferring data to and from GPU memory
2. Computations are massively parallel- the computations can be broken down into hundreds or thousands of independent units of work

GPU Acceleration Checklist



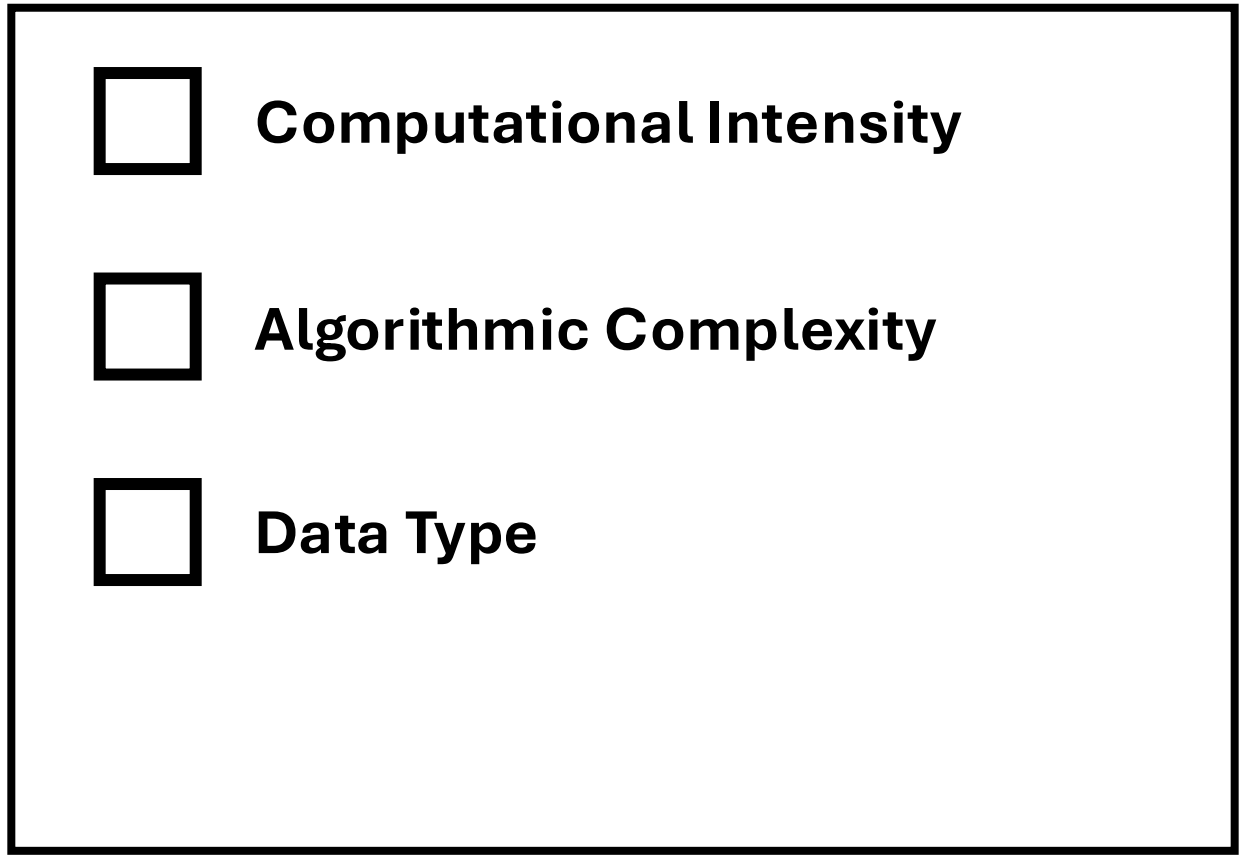
GPU Acceleration Checklist



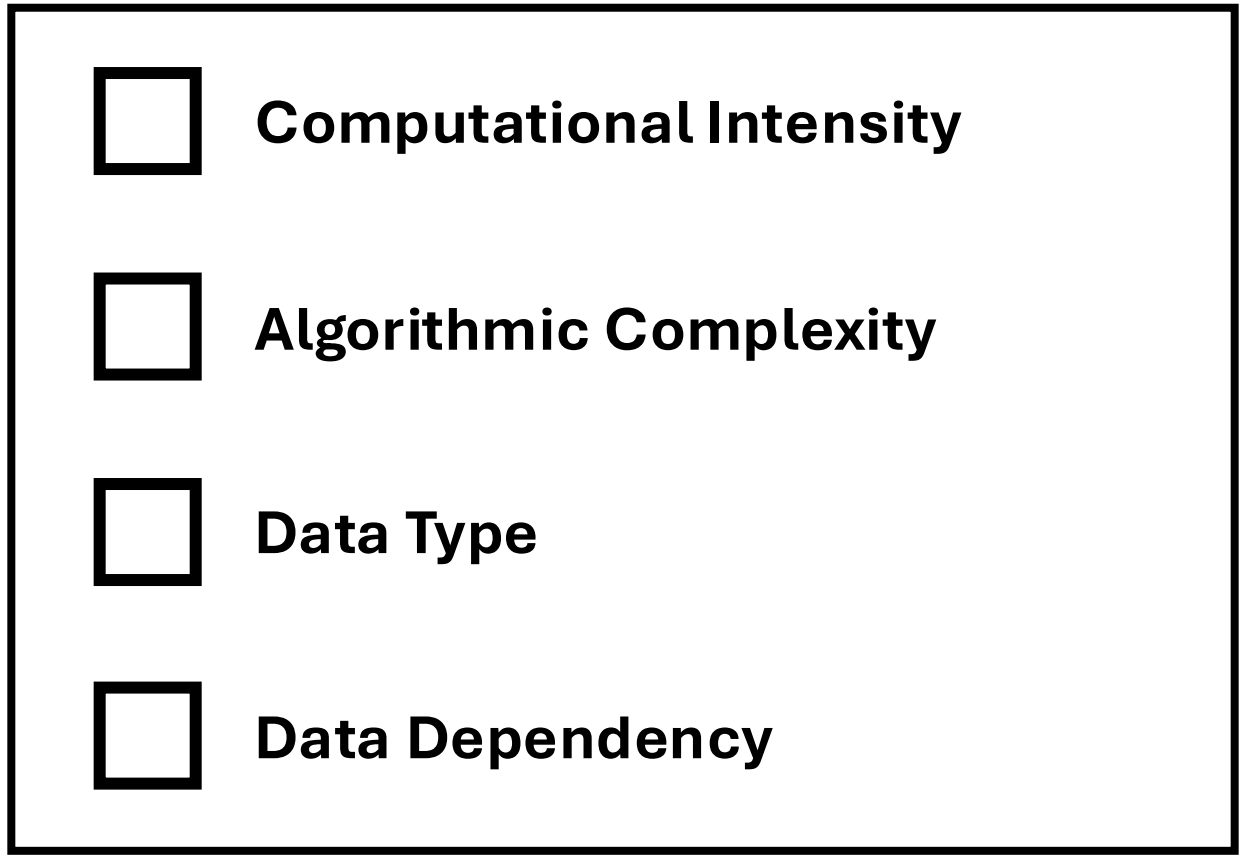
GPU Acceleration Checklist

- ☐ **Computational Intensity**
- ☐ **Algorithmic Complexity**

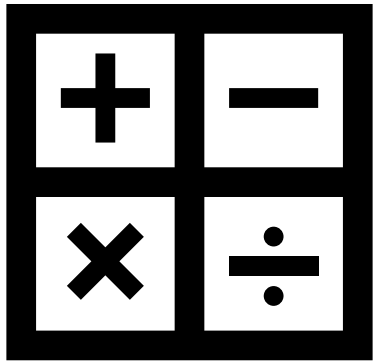
GPU Acceleration Checklist



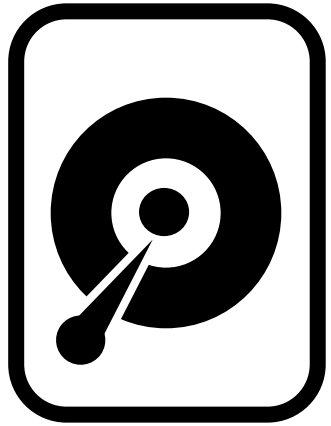
GPU Acceleration Checklist



Computational Intensity



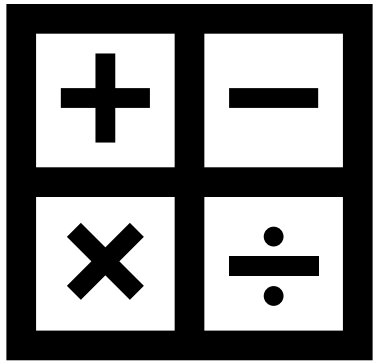
VS



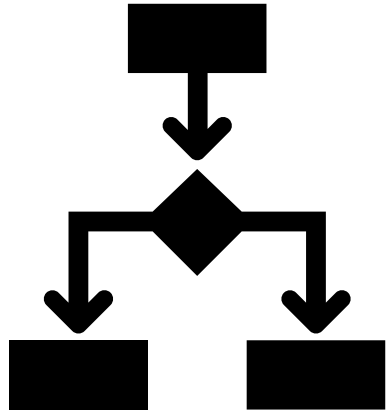
Calculations **Data Access**

GPUs perform best when processing data compared to loading and storing data (FLOP per Byte ratio)

Algorithmic Intensity



VS



Calculations **Branching Logic**

Simple code is better ported to GPUs.

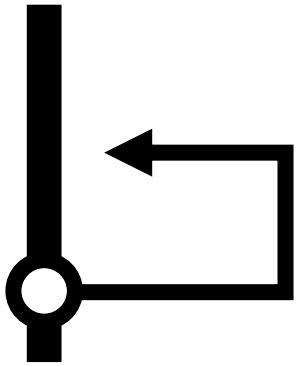
Data Type

123 vs **“Text”**

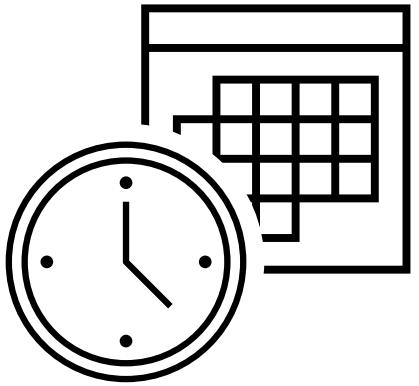
Numeric **Complex Objects**

Complex data reduces processing speed.

Data Dependency



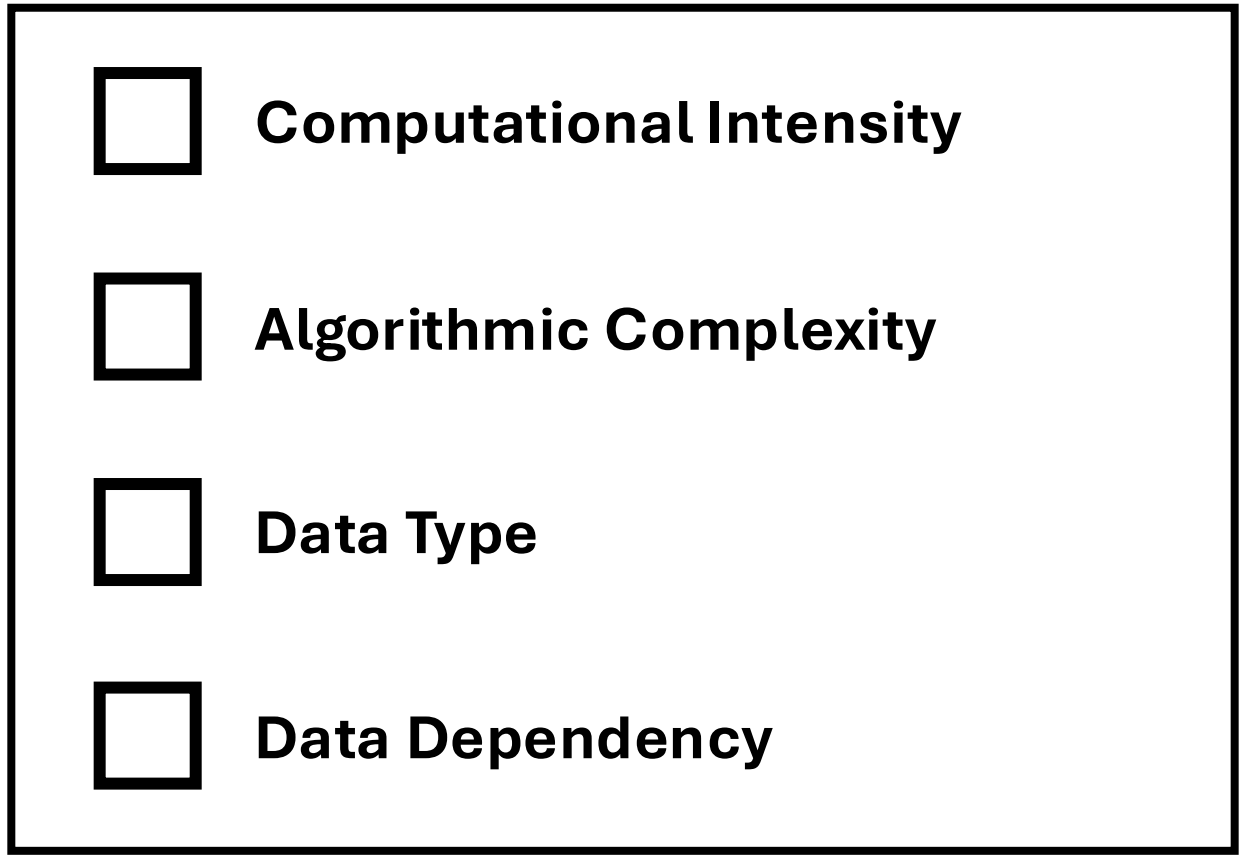
Recursion



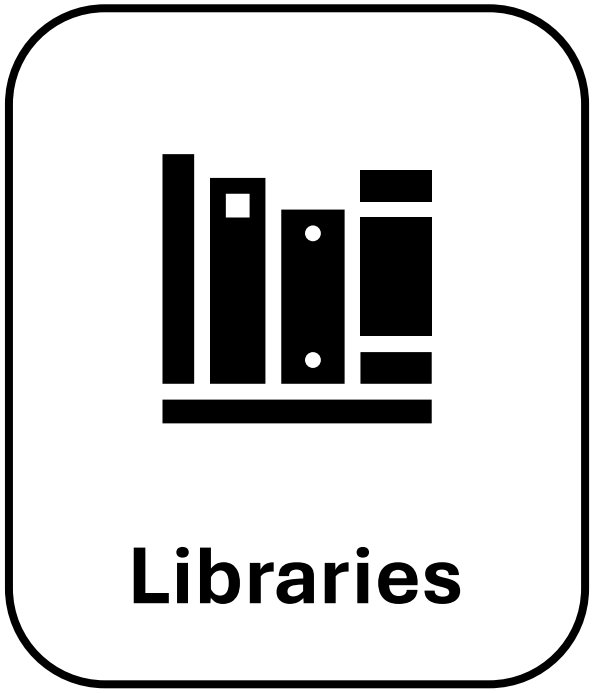
Temporal
Time / Dates

Sequential instructions or data limits parallel processing

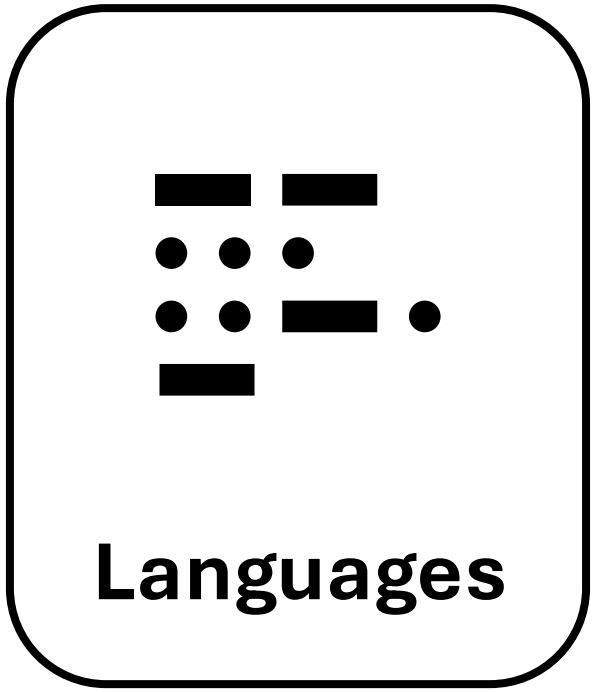
GPU Acceleration Checklist



Code Optimization







GPU Icon

Key Terms

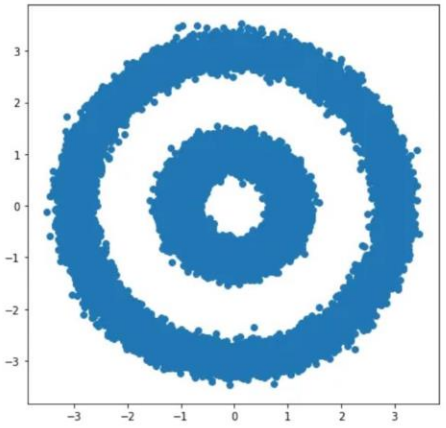
- Host == CPU
- Device == GPU
- Kernel == Functions launched on GPU

GPU Libraries – Drop in Replacement

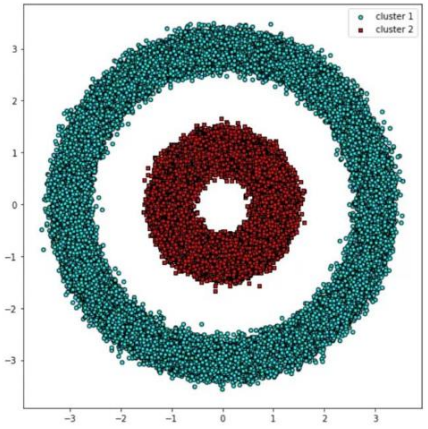
```
#create dataset with 100,000 points
from sklearn.datasets import make_circles
X, y = make_circles(n_samples=int(1e5), factor=.35, noice=.05)

#run DBSCAN clustering algorithm
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.6, min_samples=2)
y_db = db.fit_predict(X)
```

Dataset



Clusters



GPU Libraries – Drop in Replacement

```
#create dataset with 100,000 points
from sklearn.datasets import make_circles
X, y = make_circles(n_samples=int(1e5), factor=.35, noice=.05)
```

```
#run DBSCAN clustering algorithm
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.6, min_samples=2)
y_db = db.fit_predict(X)
```

GPU Libraries – Drop in Replacement

```
#create dataset with 100,000 points
from sklearn.datasets import make_circles
X, y = make_circles(n_samples=int(1e5), factor=.35, noice=.05)

#convert dataset to Pandas DataFrame
```

```
#run DBSCAN clustering algorithm
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.6, min_samples=2)
y_db = db.fit_predict(X)
```

GPU Libraries – Drop in Replacement

```
#create dataset with 100,000 points
from sklearn.datasets import make_circles
X, y = make_circles(n_samples=int(1e5), factor=.35, noice=.05)

#convert dataset to Pandas DataFrame
import pandas as pd
import cudf
X_df = pd.DataFrame({‘fea%d’%i: X[:,i] for i in range(X.shape[1])})
X_gpu = cudf.DataFrame.from_pandas(X_df)

#run DBSCAN clustering algorithm
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.6, min_samples=2)
y_db = db.fit_predict(X)
```

GPU Libraries – Drop in Replacement

```
#create dataset with 100,000 points
from sklearn.datasets import make_circles
X, y = make_circles(n_samples=int(1e5), factor=.35, noice=.05)

#convert dataset to Pandas DataFrame
import pandas as pd
import cudf
X_df = pd.DataFrame({‘fea%d’%i: X[:,i] for i in range(X.shape[1])})
X_gpu = cudf.DataFrame.from_pandas(X_df)

#run DBSCAN clustering algorithm
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.6, min_samples=2)
y_db = db.fit_predict(X)
```



```
#run GPU-accelerated DBSCAN
from cuml import DBSCAN
```

GPU Libraries – Drop in Replacement

```
#create dataset with 100,000 points
from sklearn.datasets import make_circles
X, y = make_circles(n_samples=int(1e5), factor=.35, noice=.05)

#convert dataset to Pandas DataFrame
import pandas as pd
import cudf
X_df = pd.DataFrame({‘fea%d’%i: X[:,i] for i in range(X.shape[1])})
X_gpu = cudf.DataFrame.from_pandas(X_df)

#run DBSCAN clustering algorithm
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.6, min_samples=2)
y_db = db.fit_predict(X)
```



```
#run GPU-accelerated DBSCAN
from cuml import DBSCAN
```

GPU Libraries – Drop in Replacement

```
#create dataset with 100,000 points
from sklearn.datasets import make_circles
X, y = make_circles(n_samples=int(1e5), factor=.35, noice=.05)
```

1

```
#convert dataset to Pandas DataFrame
import pandas as pd
import cudf
X_df = pd.DataFrame({‘fea%d’%i: X[:,i] for i in range(X.shape[1])})
X_gpu = cudf.DataFrame.from_pandas(X_df)
```

2

```
#run GPU-accelerated DBSCAN
from cuml import DBSCAN
db = DBSCAN(eps=0.6, min_samples=2)
y_db = db.fit_predict(X)
```

GPU-Enabled Frameworks









GPU Compiler Directives

OpenACC

GPU Compiler Directives

OpenACC

Kernel directives

Generate parallel accelerator kernels
for the loop following the directive.

GPU Compiler Directives

OpenACC

Kernel directives

Generate parallel accelerator kernels for the loop following the directive.

```
//Hello_World_OpenACC.c
void Print_Hello_World()
{
  #pragma acc kernels
  for(int i=0; i<5; i++)
  {
    printf("Hello World!\n")
  }
}
```

GPU Compiler Directives

OpenACC

Kernel directives

Generate parallel accelerator kernels for the loop following the directive.

Data directives

Generate code to manage specific data operations to support parallelism

```
//Hello_World_OpenACC.c
void Print_Hello_World()
{
  #pragma acc kernels
  for(int i=0; i<5; i++)
  {
    printf("Hello World!\n")
  }
}
```

GPU Compiler Directives

OpenACC

Kernel directives

Generate parallel accelerator kernels for the loop following the directive.

```
//Hello_World_OpenACC.c
void Print_Hello_World()
{
  #pragma acc kernels
  for(int i=0; i<5; i++)
  {
    printf("Hello World!\n")
  }
}
```

Data directives

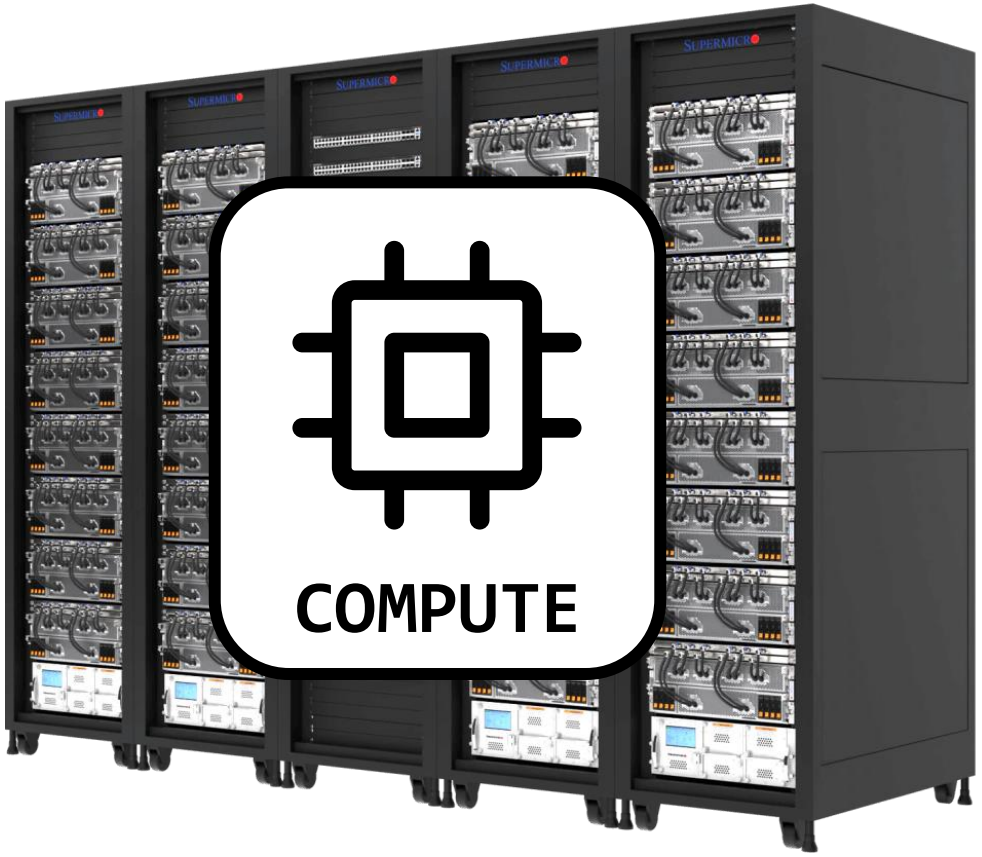
Generate code to manage specific data operations to support parallelism

```
//Hello_World_OpenACC.c
#pragma acc data copy(a)
{
  #pragma acc kernels
  for(int i=0; i<5; i++)
  {
    printf("Hello World!\n")
  }
}
```

Languages

- OpenCL (NVIDIA, AMD, & CPUs)
  - Flexible / portable option
- HIP (AMD -> NVIDIA)
  - AMD developed
  - Can convert CUDA code via `hippify`
- CUDA (NVIDIA only)
  - Most robust and largest developer community

Alpine Partitions



Alpine Partitions

acpu

General Usage



Alpine Partitions

acpu

General Usage

amem

High Memory



Alpine Partitions

acpu

General Usage

amem

High Memory



aa100

al40

ah200

artxpro6000

Nvidia GPUs

Alpine Partitions

acpu

General Usage

amem

High Memory



aa100

al40

ah200

artxpro6000

Nvidia GPUs

ami100

AMD GPUs

Alpine GPUs

	NVIDIA			AMD
Type	**A100**	**L40**	**GH200**	**MI100**
Cores	**7k**	**15K**	**17k**	**7.7k**
VRAM	**40 / 80**	**48**	**96**	**32**
Purpose	**General**	**Viz, AI Inference**	**AI Training, High Data I/O**	**Scientific**

Alpine GPUs

	NVIDIA	
Type	**H200**	**RTX Pro 6000**
Cores	**17k**	**25K**
VRAM	**141**	**96**
Purpose	**Scientific Precision**	**AI Training,**

Alpine MIG-GPUs

	ah200		
GPU Type	**h200**	**h200_3g.71gb**	**h200_2g.35gb**
VRAM	**141**	**71**	**35**
Max Cores Per GPU	**32**	**16**	**10**
Billing Weight	**370**	**185**	**123**

Alpine MIG-GPUs

	artxpro6000		
GPU Type	**rtx_pro_6000**	**rtx_pro_6000_2g.48gb**	**rtx_pro_6000_1g.24gb**
VRAM	**96**	**48**	**24**
Max Cores Per GPU	**32**	**16**	**8**
Billing Weight	**260**	**130**	**65**

Requesting GPUs

SLURM Directives:

--partition= < >

--gres=gpu:<#>

--ntasks=<#>

Requesting GPUs

SLURM Directives:

--partition= < >

--gres=gpu:<#>

--ntasks=<#>

sinteractive --partition=ah200 --gres=gpu:ah200:2 --ntasks=20

#SBATCH <directive>

Monitoring GPU Usage

- Nvidia-smi
- rocm-smi

```
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 510.47.03    Driver Version: 510.47.03    CUDA Version: 11.6     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  NVIDIA A100-PCI...  Off  | 00000000:21:00.0 Off |                    0 |
| N/A   36C    P0    40W / 250W |      0MiB / 40960MiB |      0%      Default |
|                               |                      |             Disabled |
+-------------------------------+----------------------+----------------------+
|   1  NVIDIA A100-PCI...  Off  | 00000000:81:00.0 Off |                    0 |
| N/A   36C    P0    40W / 250W |      0MiB / 40960MiB |      0%      Default |
|                               |                      |             Disabled |
+-------------------------------+----------------------+----------------------+
|   2  NVIDIA A100-PCI...  Off  | 00000000:E2:00.0 Off |                    0 |
| N/A   37C    P0    40W / 250W |      0MiB / 40960MiB |      0%      Default |
|                               |                      |             Disabled |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
```

Troubleshooting GPU Workflows

- Is your application and/or code GPU accelerated?

  Confirm that you installed the GPU accelerated version!

- Does your application or code support **multi-**GPU acceleration?

- Is your application ROCM- or CUDA-aware?

  You can't run CUDA code on AMD GPUs. Not all applications are available for AMD GPUs.

- Can your application "see" the GPU?

- Did you request enough CPUs and RAM?

seff

```
$ seff <jobid>
```

Job Overview

Job ID: 20522520

Cluster: alpine

User/Group: ralphie/ralphiegrp

State: COMPLETED (exit code 0)

Nodes: 1 | Cores per node: 21

seff

```
$ seff <jobid>
```

CPU Metrics

```
Cpu Metrics
CPU Utilized: 7-14:11:13
CPU Efficiency: 75.77% of 10-00:27:21 core-walltime
Job Wall-clock time: 11:27:01
Memory Utilized: 21.25 GiB
Memory Efficiency: 27.96% of 76 GiB (76 GiB/node)
```

seff

```
$ seff <jobid>
```

Did you use the requested CPUs?

CPU Metrics

CPU Metrics

CPU Utilized: 7-14:11:13

CPU Efficiency: 75.77% of 10-00:27:21 core-walltime

Job Wall-clock time: 11:27:01

Memory Utilized: 21.25 GiB

Memory Efficiency: 27.96% of 76 GiB (76 GiB/node)

seff

```
$ seff <jobid>
```

CPU Metrics

Did you use the requested CPUs?

CPU Metrics

CPU Utilized: 7-14:11:13

CPU Efficiency: 75.77% of 10-00:27:21 core-walltime

Job Wall-clock time: 11:27:01

Did you use the requested RAM?

Memory Utilized: 21.25 GiB

Memory Efficiency: 27.96% of 76 GiB (76 GiB/node)

seff

```
$ seff <jobid>
```

GPU Metrics

GPU Metrics

Number of GPUs: 1

GPU Type: l40

NOTE: GPU metric availability may vary by GPU type.

Please refer to our documentation for details: [...]

Max GPU Utilization: 41%

Max GPU Memory Utilized: 874.00 MiB

seff

```
$ seff <jobid>
```

GPU Metrics

```
———————— GPU Metrics ————————

Number of GPUs: 1

GPU Type: l40

NOTE: GPU metric availability may vary by GPU type.

    Please refer to our documentation for details: [...]

Max GPU Utilization: 41%

Max GPU Memory Utilized: 874.00 MiB
```

Did you use the requested GPU(s)?

seff

```
$ seff <jobid>
```

GPU Metrics

——— GPU Metrics ———

Number of GPUs: 1

GPU Type: l40

NOTE: GPU metric availability may vary by GPU type.

Please refer to our documentation for details: [...]

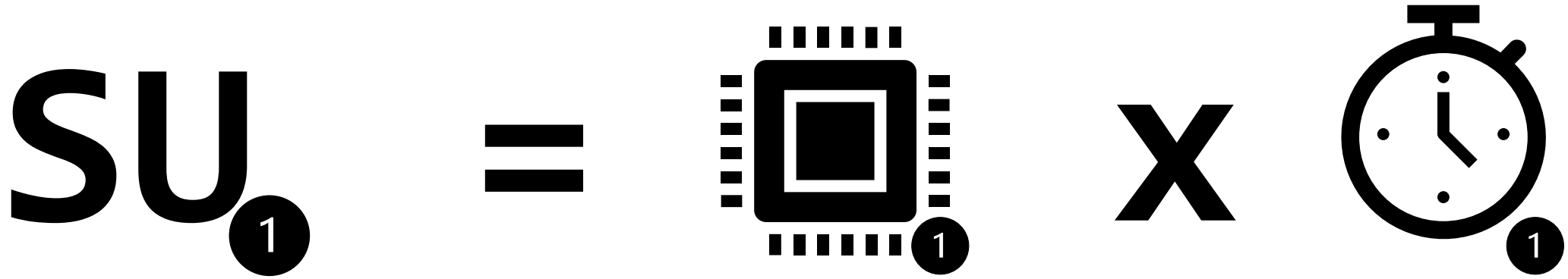
Did you use the requested GPU(s)?

Max GPU Utilization: 41%

Did you pick the *right size* GPU?

Max GPU Memory Utilized: 874.00 MiB

Service Units



Service units (SUs), sometimes called “core hours”, reflect the processing that a core performs in one hour modified by some scaling factor

suuser

How many Service Units (SUs) have I used?

```
$ suuser <username>
```

```
SU used by user ralphie in the last 30 days:
Cluster |Account     |Login   |Proper Name|TRES Name|Used|
alpine  |ucb-general|ralphie|R. Buffalo |billing  |283 |
```

How do Service Units Impact Priority?

How do Service Units Impact Priority?

Level FairShare

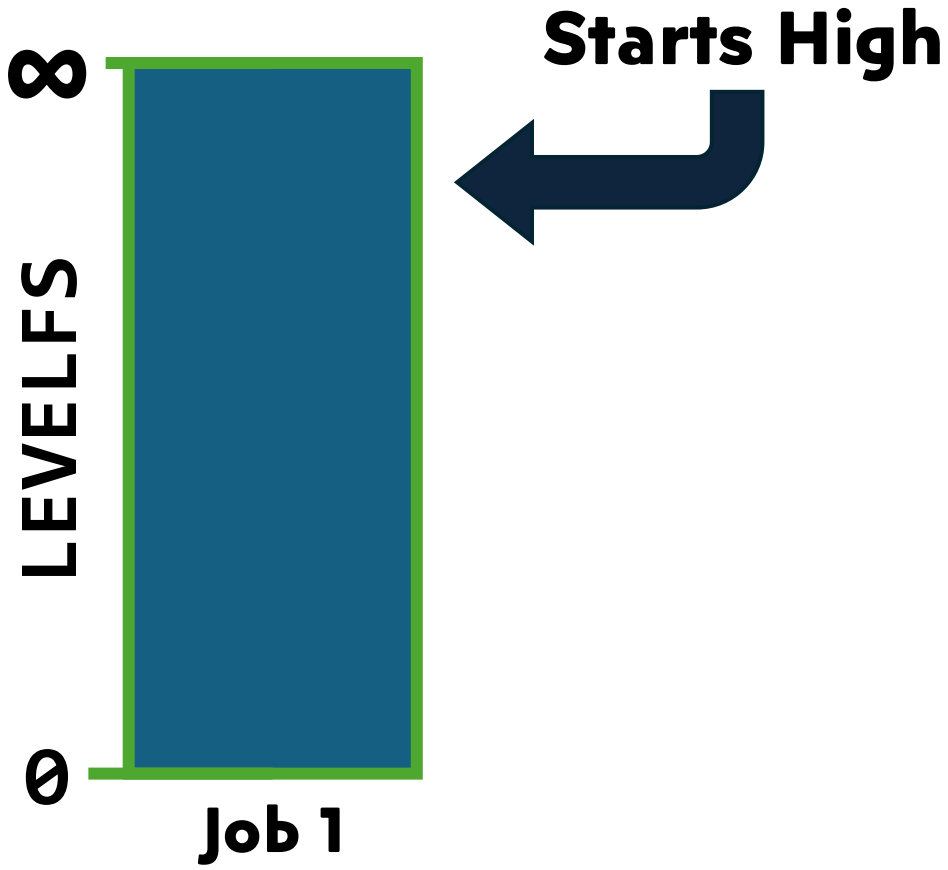
Level FairShare Calculation

$$\text{LevelFS} = \frac{\text{Shares}}{\text{Usage}}$$

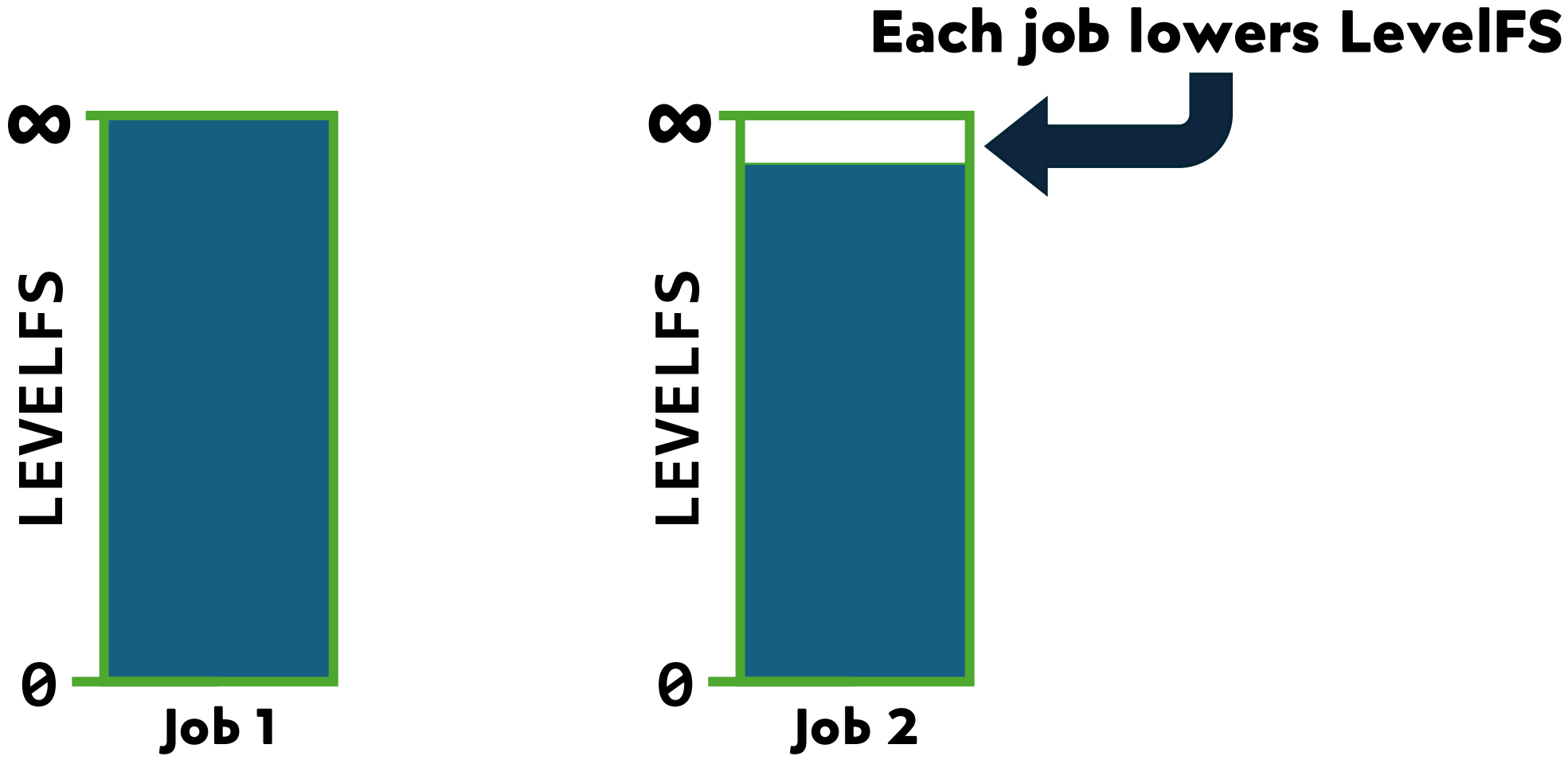
Level FairShare Calculation

$$\text{LevelFS} = \frac{\text{SU Allocated}}{\text{SU Spent}}$$

Level FairShare



Level FairShare



Level FairShare



levelfs

```
$ levelfs <username>
```

```
LevelFS for user ralphie and institution ucb:

Account              LevelFS_User         LevelFS_Inst
-------------------------------------------------------
ucb-general          2.015951             0.943096
```

> 1

High Priority

< 1

Low Priority

Documentation



https://curc.readthedocs.io/en/latest/